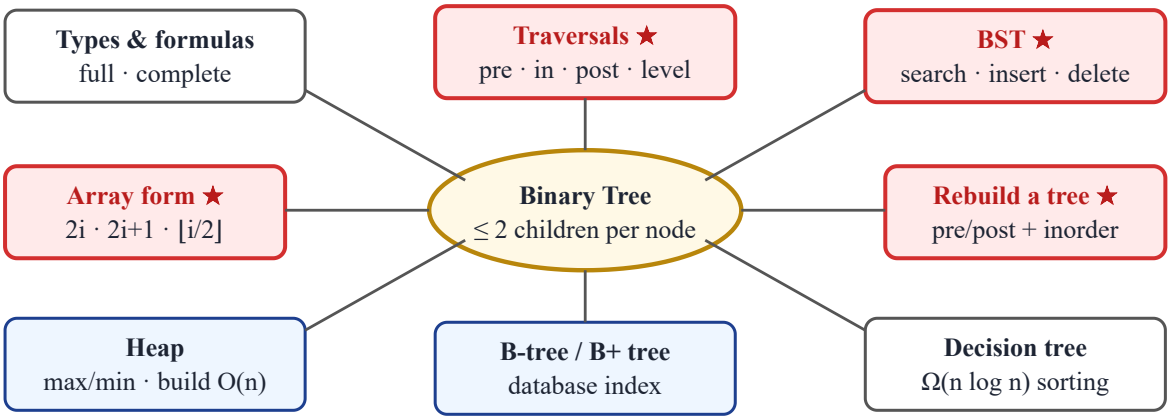
Tree Workshop: Build, Traverse, BST & Heap

Traversals · BST · array form · rebuilding · heap · B-tree: exam-ready in 11 sheets



Legend: **yellow**=key term · ●=must-memorise · =memory trick · =exam trap · red node = node in focus

Contents

1. Terminology, types & node formulas … p.2
2. Traversals & expression trees … p.3
3. BST: search & insert … p.4
4. BST: delete (3 cases) … p.5
5. Array representation … p.6
6. Rebuilding a tree I: with inorder … p.7
7. Rebuilding a tree II: pre + post … p.8
8. Heap & B-tree in one sheet … p.9
9. Decision tree lower bound + night-before … p.10
0. Practice MCQs + written answer … p.11

Syllabus coverage: "… **trees** …; Advanced: **heaps**, Fibonacci heaps, **B-trees**." Deeper heap/B-tree internals: *Heaps & B-Trees* lesson.

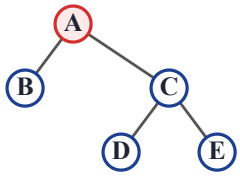
High-yield list

① BST inorder = sorted
② Postorder deletes a tree
③ 0-indexed left = 2i+1
④ Max nodes = 2^(h+1)−1
⑤ Full tree: L = I + 1
⑥ 2 children → successor
⑦ Sorting ≥ Ω(n log n)

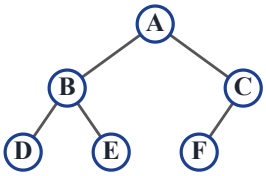
1 Terminology, Binary Tree Types & Node Formulas ★ MCQ formulas

Tree: A **hierarchical, non-linear** structure with one **root** and no cycles; n nodes have **n − 1 edges** (file system, HTML DOM). A **binary tree**: every node has **at most 2** children, *left* and *right* (a lone left child ≠ a lone right child).

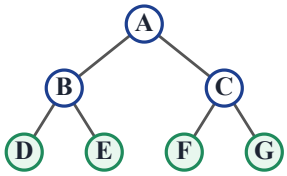
Term	Meaning	Term	Meaning
Root	only node with no parent	**Leaf**	no children (external node)
Internal node	has ≥ 1 child	Degree	number of children
Depth / level	edges from the root (root = 0)	**Height**	edges on the longest root-to-leaf path



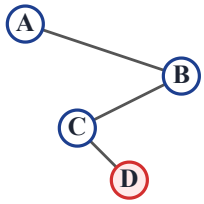
Full / proper
0 or 2 children



Complete
last level left-filled



Perfect
leaves on one level



Skewed
1 child each

Quantity (height h in edges, root at level 0)	Formula	Check
Max nodes in a tree of height h (level i holds ≤ 2^i)	**2^(h+1) − 1**	h = 3 → 15
Max nodes in N levels (root = level 1)	2^N − 1	N = 4 → 15
Min nodes in a tree of height h	h + 1 (skewed)	h = 3 → 4
Min height with n nodes = height of a complete tree	**$\lfloor \log_2 n \rfloor$**	n = 10 → 3
Leaves vs 2-child nodes (any binary tree)	**$n_0 = n_2 + 1$**	perfect h=2: 4 = 3 + 1
Full tree with I internal nodes	L = I + 1, n = 2I + 1	I = 10 → L = 11
Shapes with n nodes (Catalan)	(2n)! / ((n+1)! n!)	n = 3 → 5

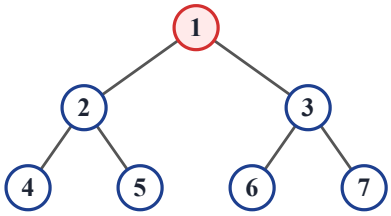
Proof (written favourite): **In a full (proper) binary tree, leaves = internal nodes + 1.** Let I = internal nodes, L = leaves. Count edges two ways: each non-root node has one parent edge, so edges = I + L − 1; each internal node has 2 children, so edges = 2I. Hence 2I = I + L − 1, i.e. **L = I + 1**. ■

Exam trap: **Height convention.** Here height counts **edges** (lone root = 0). Some books count **nodes** (lone root = 1), so "N levels" gives 2^N − 1. State your convention.

Memory trick: Levels double: 1, 2, 4, 8 …, so h+1 full levels hold "next power of two, minus one". Perfect ⊂ complete and perfect ⊂ full, but full ⇏ complete.

2 Traversals: Pre, In, Post & Level Order ★ Most tested

Traversal: Visit every node once. The 3 **depth-first** orders differ only in **when the root is visited**; **level order** is breadth-first (queue). All O(n).



Preorder (Root L R): 1 2 4 5 3 6 7

Inorder (L Root R): 4 2 5 1 6 3 7

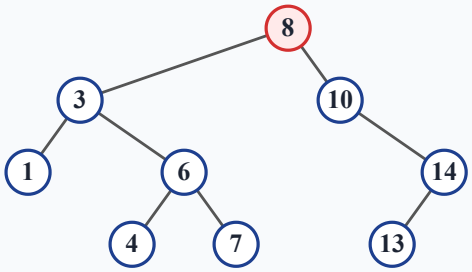
Postorder (L R Root): 4 5 2 6 7 3 1

Level order (queue): 1 2 3 4 5 6 7

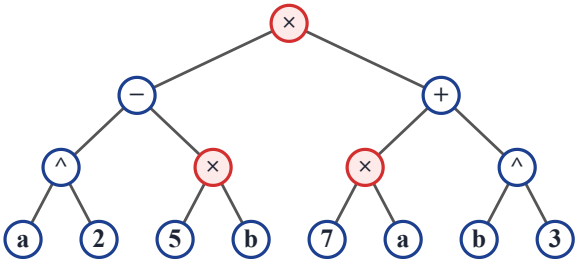
root 1 is first in pre, middle in in, last in post

Traversal	Order	Typical use
Preorder	Root L R	copy / serialise a tree, prefix (Polish) form
Inorder	L Root R	BST keys in **ascending** order, infix form
Postorder	L R Root	**delete/free** a tree, postfix evaluation, folder sizes
Level order	by levels (queue)	BFS, print by levels, heap array order

Worked example: Insert **8 3 10 1 6 14 4 7 13** into an empty BST and traverse it in every order, including the unusual **Right → Left → Root**.



Pre: **8 3 1 6 4 7 10 14 13**
In: **1 3 4 6 7 8 10 13 14** (sorted ✓)
Post: **1 4 7 6 3 13 14 10 8**
Level: **8 3 10 1 6 14 4 7 13**
R → L → Root: **13 14 10 7 4 6 1 3 8** *(= preorder reversed)*



Expression tree for X = (a² − 5b) × (7a + b³): operators inside, operands at the leaves.
Prefix (pre): × − ^ a 2 × 5 b + × 7 a ^ b 3
Postfix (post): **a 2 ^ 5 b × − 7 a × b 3 ^ + ×**

Memory trick: **Pre / In / Post = where the Root goes**. Walk round the outline; write a node at its **left** side (pre), **bottom** (in) or **right** (post).

Exam trap: Child deleted before its parent → **postorder**. Inorder is sorted **only for a BST**.

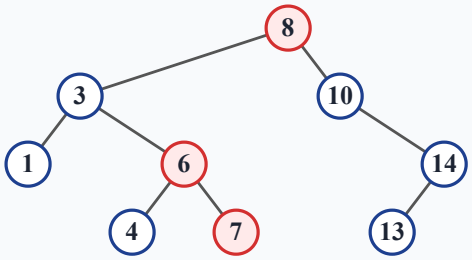
3 Binary Search Tree: Search & Insert ★ Bank & PSU written

BST property: For **every** node: all keys in its left subtree are **smaller**, all keys in its right subtree are **larger** (no duplicates, unless you state a rule such as "equal goes right"). Consequence: **inorder = sorted**.

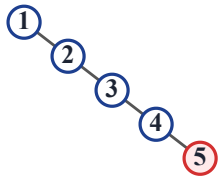
```
/* search: iterative */
node* search(node* t, int k) {
  while (t && t->key != k)
    t = (k < t->key) ? t->left
                     : t->right;
  return t;   /* NULL = absent */
}
```

```
/* insert: recursive */
node* insert(node* t, int k) {
  if (!t) return newNode(k);
  if (k < t->key)
    t->left = insert(t->left, k);
  else if (k > t->key)
    t->right = insert(t->right, k);
  return t;   /* duplicate ignored */
}
```

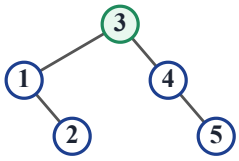
Worked example: Insert **8 3 10 1 6 14 4 7 13** into an empty BST: each key walks down from the root, left if smaller, right if larger, until it meets an empty link.



4: 4 < 8, 4 > 3, 4 < 6 → left of 6
13: 13 > 8, 13 > 10, 13 < 14 → left of 14
search 7: 8, 3, 6, 7 → found, 4 comparisons
search 5: 8, 3, 6, 4, then NULL
→ not found after 4 comparisons;
5 would be inserted right there (right of 4)



keys 1 2 3 4 5: h = 4



keys 3 1 4 2 5: h = 2

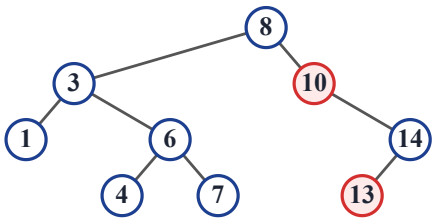
Every operation is **O(h)**
balanced: $h \approx \log_2 n \rightarrow O(\log n)$
skewed: $h = n - 1 \rightarrow O(n)$
fix: AVL / red-black trees

Memory trick: **"Smaller left, larger right, stop at NULL."** The **first** key is always the root; sorted input turns a BST into a linked list.

Exam trap: A BST is **not** automatically balanced: search is **O(log n)** only when balanced and **O(n)** in the worst case. Min = leftmost node, max = rightmost.

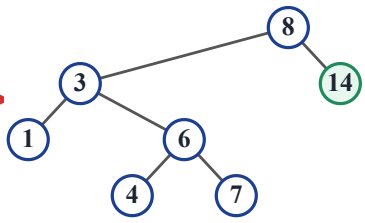
4 BST Delete: the Three Cases ★ Written favourite

1. **Leaf**: just remove it. Delete 13 → 14 loses its left child.
2. **One child**: link the child to the parent. Delete 10 → 14 moves up.
3. **Two children**: copy in the **inorder successor** (smallest key of the right subtree), then delete that successor (it has ≤ 1 child). The **predecessor** (largest on the left) also works.

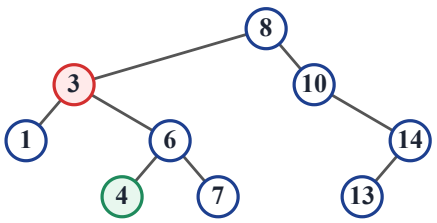


13 = leaf, 10 = one child

delete 13, 10

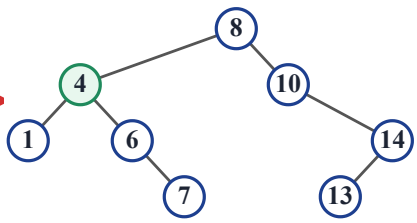


14 moves up



successor of 3 = 4

delete 3



inorder 1 4 6 7 8 10 13 14 ✓

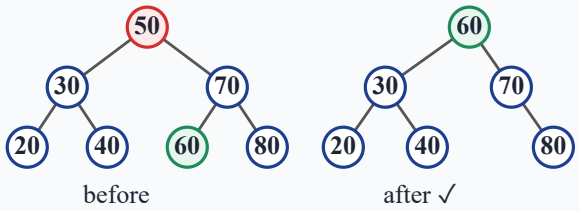
Worked example 1: Delete the **root 8**: successor = **10** (no left child), so 10 moves up and 14 takes its place → 10(3(1, 6(4, 7)), 14(13)). Using the predecessor, **7** becomes the root.

Worked example 2: a full written question: (a) Insert 50, 30, 70, 20, 40, 60, 80. (b) Print the postorder. (c) Delete 50 and redraw.

(a) 50 root; 30, 70; then 20, 40 and 60, 80.

(b) Postorder: **20 40 30 60 80 70 50**

(c) Successor = **60**: copy it up, remove the leaf.



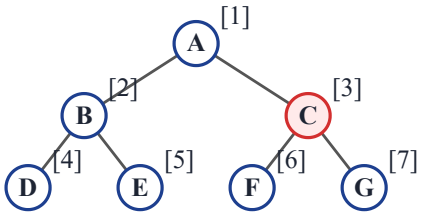
Memory trick: Successor = **"one right, then all the way left"**; predecessor = **"one left, then all the way right"**. Then check the inorder is still sorted.

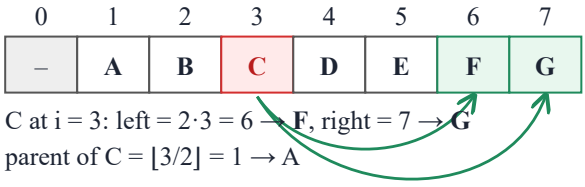
Exam trap: In case 3 you copy the successor's **key**; "replace with the right child" is wrong when that child has a left subtree (deleting 3: 6 has left child 4).

5 Storing a Binary Tree in an Array ★ Written + BCS MCQ

Sequential representation: Put the root at index 1 and number the positions in **level order**, left to right. A missing node leaves an empty slot; index arithmetic replaces pointers.

	Root at	Left child of i	Right child of i	Parent of i
1-indexed	1	**2i**	2i + 1	⌊i / 2⌋
0-indexed (C, Java)	0	**2i + 1**	2i + 2	⌊(i − 1) / 2⌋





Worked example: a tree with gaps: The BST of sheet 4: 14 = right of 10 → 2·3+1 = 7; 13 = left of 14 → 2·7 = 14.

1	2	3	4	5	6	7	8	9	10	11	12	13	14
8	**3**	**10**	**1**	**6**	–	**14**	–	–	**4**	**7**	–	–	**13**

9 keys use **14 slots**; a right-skewed tree of n nodes needs $2^n - 1$.

Worked example: postorder → array: Nodes a…i, postorder **a b f c h d e g i**. Assume a **complete** tree (one traversal alone is ambiguous): its positions in postorder are **8 9 4 5 2 6 7 3 1**, so 8 = a, 9 = b, 4 = f, …

1	2	3	4	5	6	7	8	9
i	**h**	**g**	**f**	**c**	**d**	**e**	**a**	**b**

tree i(h(f(a, b), c), g(d, e))
postorder a b f c h d e g i ✓

Array vs linked: Array: parent in **O(1)**, ideal for complete trees (heaps), wasteful if sparse. Linked: n + 1 NULL pointers, any shape.

Memory trick: **Double** to go down-left, **halve** to go up (1-indexed).

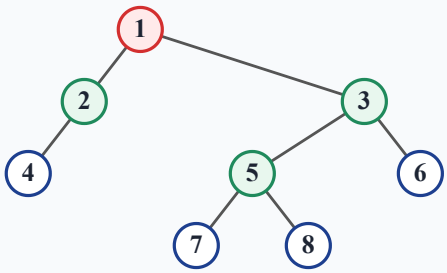
Exam trap: Read the indexing! "Zero-based, left child of i?" → **2i + 1**, not 2i. Leaves of a complete tree (1-indexed) are ⌊n/2⌋+1 … n.

6 Rebuilding a Tree from Inorder + One More ★ BUET-pattern written

The rule: **Preorder** names the root **first**; **postorder** names it **last**. **Inorder** splits the rest into left and right subtrees. Recurse. **Inorder + pre or post** = one unique tree.

Worked example 1: preorder + inorder: Inorder: **4 2 1 7 5 8 3 6** · Preorder: **1 2 4 3 5 7 8 6**

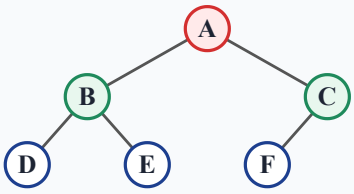
1. Pre[0] = **1** = root. Inorder: left {4 2}, right {7 5 8 3 6}.
2. Left: root **2**; inorder "4 2" → 4 is its **left**.
3. Right, pre 3 5 7 8 6: root **3**; inorder "7 5 8 | 3 | 6".
4. {7 5 8}, pre 5 7 8: root **5**, left 7, right 8; 6 is 3's right.



Read back pre ✓ and in ✓; postorder = **4 2 7 8 5 6 3 1**.

Worked example 2: postorder + inorder: Inorder: **D B E A F C** · Postorder: **D E B F C A**.

1. Root = last = **A**; left {D B E}, right {F C}.
2. Left post "D E B" → root **B**, D left, E right.
3. Post "F C" → root **C**; inorder "F C" → F is its **left**.



Preorder of the result: A B D E C F.

Worked example 3: a classic with a misprint: Preorder **a b d e c f g** with a 5-letter inorder "d e a f g": an inorder must list all 7 keys. Reading it as **d b e a f c g** gives a(b(d, e), c(f, g)), postorder **d e b f g c a**. State the assumption.

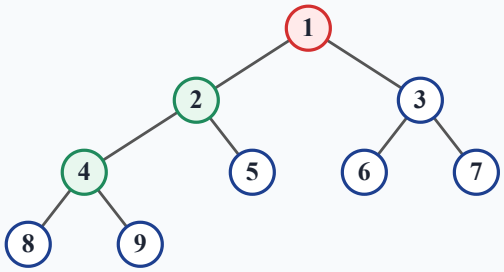
Memory trick: "**Pre/Post point, In splits.**" Pre/post say *who* is the root; inorder says *who goes where*.

7 Preorder + Postorder, and BST Shortcuts ★ Bank & BUET

Method (full trees only): Root = pre[0] = post[last]. The **next** preorder key roots the **left** subtree; in postorder, everything up to it is the left subtree, the rest the right. Recurse.

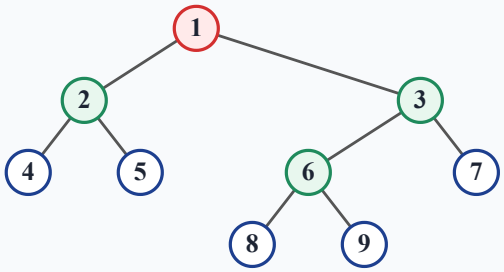
Worked example 1: Pre: **1 2 4 8 9 5 3 6 7** · Post: **8 9 4 5 2 6 7 3 1**

1. Root 1, next key **2**: post up to 2 → left {8 9 4 5 2}, right {6 7 3}.
2. Root 2, next key **4**: post up to 4 → {8 9 4}; 5 is 2's right.
3. 4(8, 9); right part 3(6, 7).



Worked example 2: Pre: **1 2 4 5 3 6 8 9 7** · Post: **4 5 2 8 9 6 7 3 1**

1. Root 1, next key 2: post up to 2 = {4 5 2} → 2(4, 5).
2. Right: pre 3 6 8 9 7 → root 3, next key 6 → 6(8, 9); 7 is 3's right.



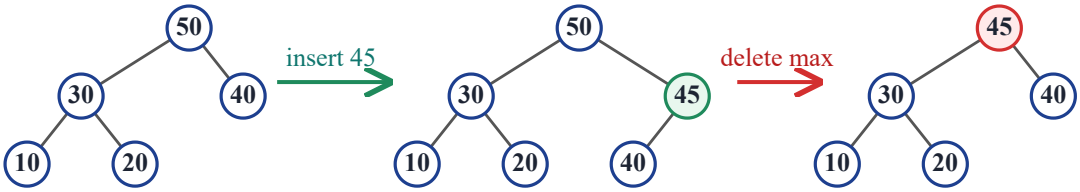
Exam trap: **Pre + post** is ambiguous once a node has one child: pre "A B", post "B A" fits B as left **or** right child. Inorder + any one other is **always unique**; one traversal alone never is.

BST shortcut: For a **BST** the inorder is the sorted keys, so preorder alone fixes it: pre 8 3 1 6 4 7 10 14 13 → sort → rebuild (or insert in that order) → the BST of sheet 4.

Drill: Pre 1 5 8 7 4 9 3 · In 8 5 7 1 9 4 3 → 1(5(8, 7), 4(9, 3)): a plain binary tree, not a BST.

8 Heap & B-tree: the Exam Essentials ★ Priority queue · DB index

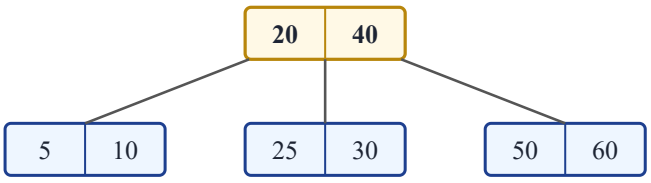
Heap: A **complete** binary tree in an array: **max-heap** parent ≥ children (root = max), **min-heap** parent ≤ children. Siblings are unordered.



[50 30 40 10 20] [50 30 45 10 20 40] [45 30 40 10 20]

1. **Insert 45** at index 6; parent $\lfloor 6/2 \rfloor = 3$ holds $40 < 45 \rightarrow$ swap; parent $50 \geq 45 \rightarrow$ stop (**sift-up**, O(log n)).
2. **Delete max**: remove 50, move the last key 40 to the root; child $45 > 40 \rightarrow$ swap (**sift-down**, O(log n)).
3. **Build-heap**: sift-down from node $\lfloor n/2 \rfloor$ back to 1: total **O(n)**, not O(n log n). Find-max is **O(1)**; heapsort O(n log n).

B-tree of order m: A **self-balancing multi-way search tree**: $\leq$ **m** children and $\leq$ **m − 1** keys per node; non-root internal nodes $\geq$ **⌈m/2⌉** children; all leaves on the **same level**. Height = $O(\log_m n)$.



Order 3: ≤ 3 children, ≤ 2 keys
keys < 20 | 20…40 | > 40
node = one disk block
all leaves on one level ✓
full node → split, middle up

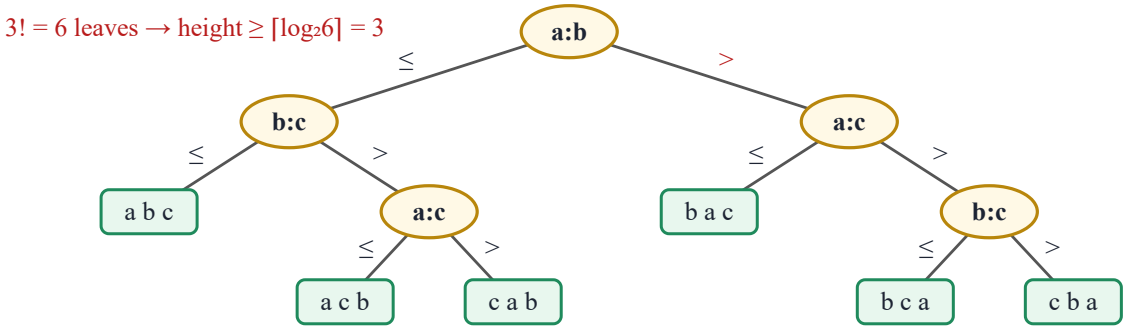
Why databases use B-trees: Node = disk block with hundreds of keys: millions of rows in 3–4 levels = 3–4 reads. **B+ tree**: records only in the linked leaves, so **range queries** are a leaf scan (MySQL InnoDB, PostgreSQL).

Exam trap: **A heap is not a BST**: searching a heap is O(n). "Order m" = *max m children* (Knuth); CLRS uses *minimum degree t*. Say which.

Memory trick: **Heap = priority, BST = search, B-tree = disk.** Insert sifts **up**, delete sifts **down**.

9 Decision Tree: Why Sorting Needs Ω(n log n) ★ BCS MCQ

Decision tree model: Any **comparison sort** on n keys is a binary tree: internal node = one comparison ($a \leq b$?), **leaf** = one final ordering, one run = one root-to-leaf path. So **worst-case comparisons = height**.



1. n distinct keys have **n!** orderings, each needs its own leaf → leaves $\geq n!$.

2. A binary tree of height h has at most **2^h** leaves → 2^h ≥ n! → **$h \geq \log_2(n!)$**.
3. $n! \geq (n/2)^{(n/2)} \Rightarrow \log_2(n!) \geq (n/2)\cdot\log_2(n/2) = \mathbf{\Omega(n \log n)}$. Merge sort and heapsort are optimal.

Exam trap: Counting/radix sort are **not** comparison sorts: O(n + k) is possible. In ML a "decision tree" is a **classifier**.

Numbers: $\lceil \log_2 n! \rceil$: n = 3 → 3, n = 4 → **5**, n = 5 → 7. Search in a sorted array: Ω(log n).

Night-before sheet:

- n nodes → n − 1 edges, n + 1 NULL links
- max nodes height h = 2^(h+1) − 1; N levels = 2^N − 1
- full tree: L = I + 1; any tree: $n_0 = n_2 + 1$
- complete tree height = $\lfloor \log_2 n \rfloor$
- Pre = Root L R · In = L Root R · Post = L R Root
- BST inorder = sorted; postorder deletes a tree
- level order uses a queue; DFS uses a stack
- R-L-Root = preorder reversed
- BST ops O(h): balanced O(log n), skewed O(n)
- delete, 2 children → inorder successor
- 1-indexed: 2i, 2i+1, ⌊i/2⌋; 0-indexed: 2i+1, 2i+2
- inorder + pre/post → unique tree
- heap: insert/delete O(log n), build O(n)
- B-tree: all leaves same level, node = disk block
- sorting lower bound: $h \geq \log_2 n! = \Omega(n \log n)$
- 4 keys need ≥ 5 comparisons

Memory trick: **"n! leaves, 2^h room"**: a tree with 2^h slots must fit n! answers, so $h \geq \log_2 n!$.

✓ Practice MCQs & a Model Written Answer

1. Which traversal of a binary search tree lists the keys in ascending order?
a) Preorder b) Postorder **c) Inorder** d) Level order

2. Which method is used to prove the lower bound of comparison sorting?
a) Decision tree b) Recursion tree c) Master theorem d) Dynamic programming

3. To delete all nodes so that each child is deleted before its parent, use ___ order.
a) Pre b) In c) Level **d) Post**

4. In a zero-based array storing a complete binary tree, the left child of index i is at:
a) 2i **b) 2i + 1** c) i + 1 d) i − 1

5. Maximum number of nodes in a binary tree of height 3 (root at height 0):
a) 8 b) 7 **c) 15** d) 16

6. A full binary tree has 10 internal nodes. How many leaves does it have?
a) 9 b) 10 c) 20 **d) 11**

7. Worst-case time to search a BST of n nodes:
a) O(1) **b) O(n)** c) O(log n) d) O(n log n)

8. Building a binary heap from n unsorted keys (bottom-up) takes:
a) O(n) b) O(log n) c) O(n log n) d) $O(n^2)$

9. When a BST node with two children is deleted, its key is replaced by:
a) its parent b) its left child **c) its inorder successor** d) the root

10. Minimum number of comparisons needed in the worst case to sort 4 keys by comparisons:
a) 4 **b) 5** c) 6 d) 24

Answers: 1c 2a 3d 4b 5c 6d 7b 8a 9c 10b

Written (10 marks): A binary tree has inorder **D B H E A I F J C G** and preorder **A B D E H C F I J G**. Draw the tree and write its postorder.

Model answer:
Root = first of preorder = **A**; inorder splits left {D B H E}, right {I F J C G}. *[2]*
Left: pre B D E H → root B; inorder "D | B | H E" → D is B's left; {H E} with pre E H → E, with H as its left child. *[2]*
Right: pre C F I J G → root C; inorder "I F J | C | G" → G is C's right; {I F J} with pre F I J → F(I, J). *[2]*
Tree: A(B(D, E(H, –)), C(F(I, J), G)). *[2]* Postorder: **D H E B I J F G C A**; reading back the preorder and inorder from the drawing confirms it ✓. *[2]*

Before the exam ✓: ☐ say your height convention ☐ trace traversals with the outline walk ☐ name the successor rule in BST delete ☐ check the indexing base (0 or 1) ☐ read back both traversals after rebuilding ☐ heap ≠ BST